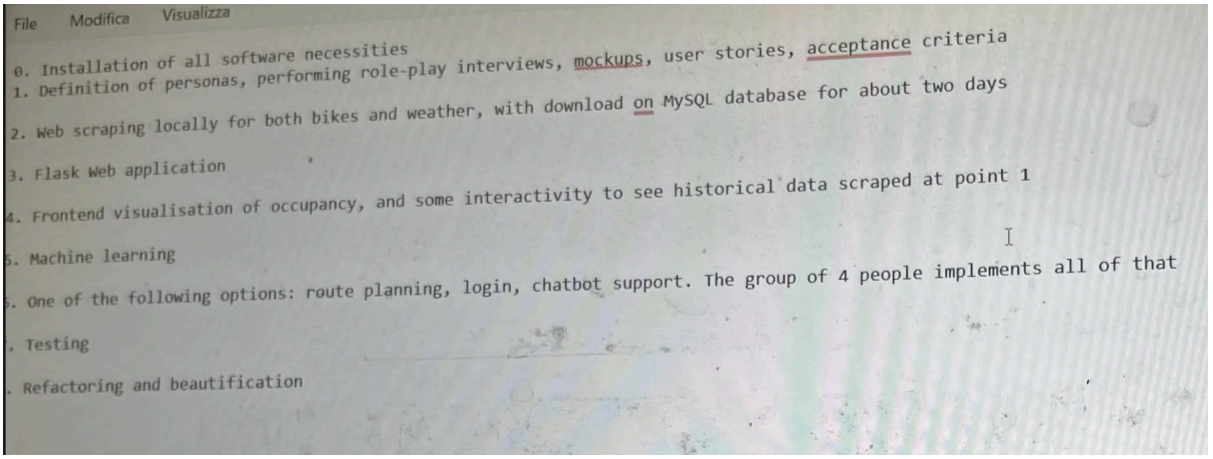
Sprint 2 - General Tasks

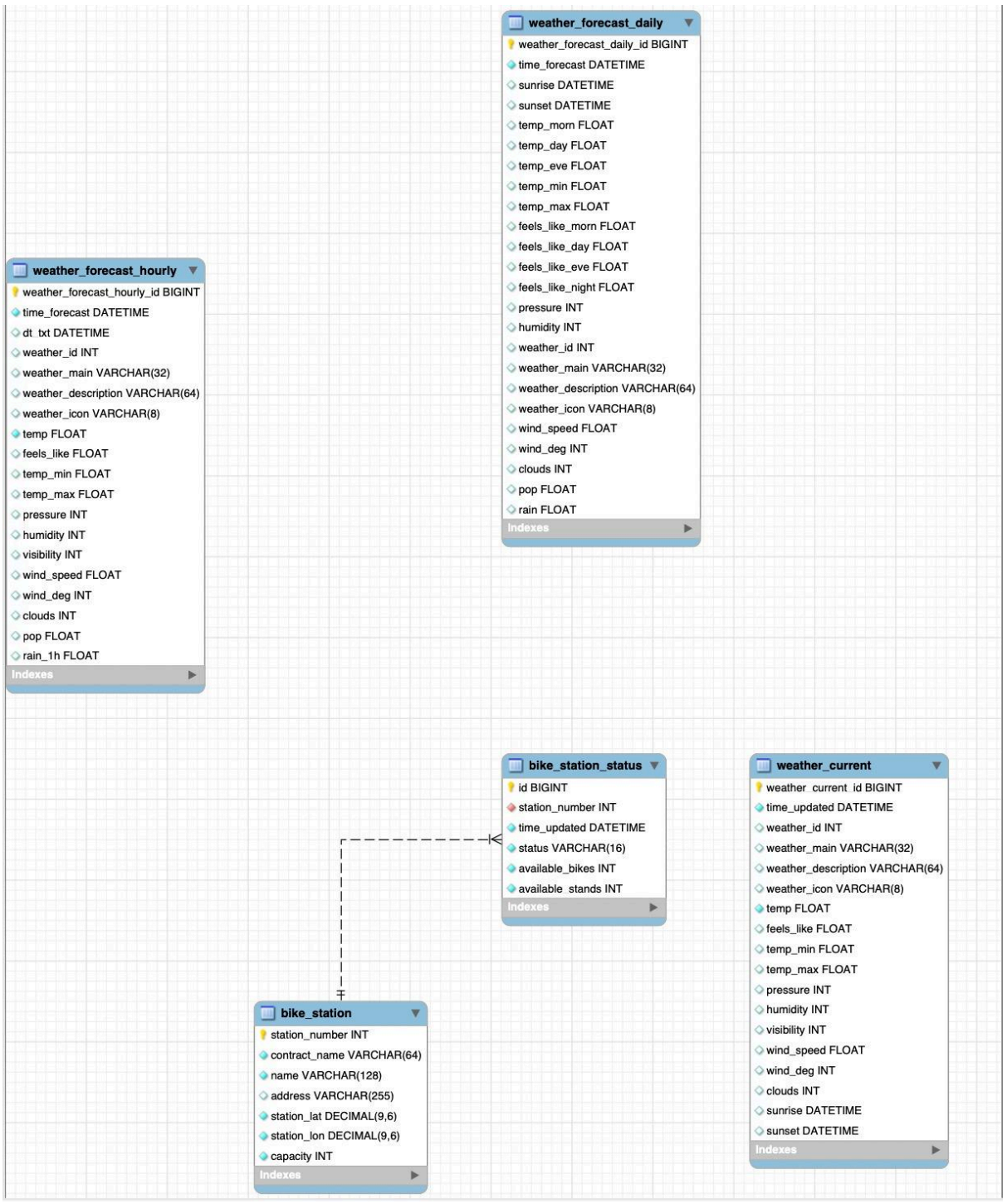
- Web scraping - bikes and weather
    - Hourly weather for current weather
    - Every 5 minutes for bikes
- Implement Flask functions to retrieve current OpenWeather and JCDecaux data and print them on the browser as JSON
- Implement Flask functions to retrieve data from your database and print them on the browser as JSON
- Execute the above on EC2, after testing on your laptop
- Define and implement a git process using feature branches to be adopted throughout the project
- Frontend visualisation
- Machine learning (?) - was told by the instructor that we can do this if we want but not necessary in this sprint
- Database design and ER tables

Goal of Sprint 2:
Implement automated scraping, database storage and a basic Flask backend capable of serving live and stored data

Additional tasks:
- Decide who will be the Scrum master for the second sprint
- Schedule regular meetings at a consistent time
- Meet outside of practicals via Zoom or in person for development work
- Submit written sprint retrospective at the end of each sprint via the web form
- Meet with assigned demonstrator every other Tuesday for sprint review, followed by sprint planning
- During sprint planning, update product backlog with user stories, bug corrections, learning activities and time estimates
- During sprint planning, create sprint backlog with task decomposition and time estimates
- Track actual time taken vs estimated time for tasks


File Modifica Visualizza

0. Installation of all software necessities
1. Definition of personas, performing role-play interviews, mockups, user stories, acceptance criteria
2. Web scraping locally for both bikes and weather, with download on MySQL database for about two days
3. Flask Web application
4. Frontend visualisation of occupancy, and some interactivity to see historical data scraped at point 1
5. Machine learning
6. One of the following options: route planning, login, chatbot support. The group of 4 people implements all of that
7. Testing
8. Refactoring and beautification



A relational database was used to organise data and maintain integrity.

- Candidate, Department, JobPosition, Skill, Interview

  Primary keys (e.g., `cand_id`, `skill_id`, `pos_id`) uniquely identify records.
  Foreign keys define relationships

- JobPosition linked to Department (`dept_id`)
- Interview linked to Candidate and JobPosition (`cand_id`, `pos_id`)

Many-to-many relationships handled using junction tables:

- Candidate_Skill (candidates ↔ skills)
- JobPosition_Skill (jobs ↔ required skills)

Database is normalised (3NF) to reduce redundancy.
Interview table stores interview date and outcome.
Design supports

- Matching candidates to jobs based on skills
- Managing departments and roles
- Tracking interview results